

Sagnik Chatterjee

+1 (413) 210-8384 | sagnikchatte@umass.edu | linkedin.com/in/sagnikchatterjee3 | github.com/sagnikc395 | sagnikc395.github.io

EDUCATION

University of Massachusetts Amherst

M.S. in Computer Science

Amherst, MA

Aug 2025 – May 2027

- GPA: **4.0/4.0** | Coursework: NLP, Machine Learning, Software Engineering

Manipal Institute of Technology

B.Tech in Computer Science & Engineering

Manipal, India

Aug 2018 – Aug 2022

TECHNICAL SKILLS

Languages: Python, Java, Go, TypeScript, SQL

Backend: REST APIs, FastAPI, Flask, Nextjs, Microservices, System Design

Data & Streaming: PySpark, Apache Spark, Apache Flink, Apache Kafka, Airflow, DBT

Databases & Warehouses: PostgreSQL, Snowflake, Redshift, Oracle, SQLite

Cloud & Infrastructure: AWS, Docker, Terraform, GitHub Actions

EXPERIENCE

Data Engineer — Backend & Data Platform

Sept 2022 – Aug 2025

IBM India Pvt. Ltd.

Gurgaon, India

- Migrated a **1.5 PB/day** pipeline from on-prem Hadoop to AWS, saving **\$20K/month**; re-engineered Pig/Hive ETL from HBase to **Apache Spark (Java)**, maintaining **400+ critical metrics** with sub-second dashboard latency and cutting on-call incidents by **40%**; backfilled **350 TB** of raw events with automated retry and validation scripts.
- Re-architected a legacy Jenkins batch ETL using **Python + Airflow**, cutting runtime from **3 hours to under 5 minutes** (**\$2K/month** saved, **60%** lower ops overhead); led a **Snowflake** POC improving query latency by **40%**, then shipped a **Snowflake + DBT** analytics framework that dropped report turnaround from **1.5 months to under 1 week** with **90% customer adoption**.
- Owned backend for a client-facing platform serving **10+ enterprise clients**; built **PySpark ETL** pipelines processing thousands of records/hour and Kibana dashboards covering **6+ ingestion pipelines**, lifting reliability from **~91% to 97%**; configured **Prometheus** alerting (12 rules) across Dockerized services, cutting **MTTD by ~40%** and **MTTR by ~25%**.
- Built an internal **GenAI** platform integrating **IBM watsonx SDK** with a **FastAPI** backend and prompt-chaining pipelines, letting non-technical associates query structured data in natural language and reducing ad-hoc engineering requests by **~40%**; architected a centralized **Redshift** KPI model with self-serve SQL data-quality checks, and delivered a **Flink** stateful streaming POC powering near real-time APIs for enterprise customers.

Data Engineer Intern

Jan 2022 – Jul 2022

IBM India Pvt. Ltd.

Gurgaon, India

- Built **ETL pipelines** ingesting **50+ GB/month** of dealer transaction data via **Snowflake, Airflow, and Python** — cut processing time by **~35%** and reduced pipeline failure rate from **~12% to under 3%**; built Kibana dashboards across **500+ dealer nodes**, cutting ops investigation time by **~30%**.

PROJECTS

deep-research | Python, Agno, MCP, Firecrawl, Hugging Face, Textual, asyncio

GitHub | Blog

- Built a **multi-agent deep research system** (plan → split → research → synthesize) over **open-source LLMs** (DeepSeek-V3, MiniMax-M1) via Hugging Face Inference; fanned out parallel researcher agents through a **ThreadPoolExecutor** with per-thread **asyncio** event loops to handle **Firecrawl over MCP**, with a deterministic Python coordinator dispatching subtasks and synthesizing results; added runtime guardrails — per-agent **tool.call_limit**, repeated tool-call **loop detection**, and per-stage / pipeline-wide **token-budget warnings**.

flowforge | Python, FastAPI, Temporal.io, PostgreSQL, Docker

GitHub | Blog

- Built a **distributed workflow engine** using **Temporal.io durable execution** and the **Saga pattern** — each activity registers a compensating action enabling **automatic rollback** (payment refund + inventory restock) on downstream failure; **idempotent async operations** ensure safe retries under worker crashes or network partitions.

weave | Python, asyncio, aiohttp, FastAPI, SQLite, MCP

GitHub | Blog

- Built a **concurrent web crawler** with **asyncio + aiohttp** for non-blocking I/O and **ProcessPoolExecutor** for CPU-bound HTML parsing, **bypassing the GIL** — benchmarked **~60%** parse-time reduction vs. thread pools; used **asyncio.Semaphore** and per-domain locks for politeness; exposed via a **FastAPI REST API** and an **MCP server for LLM tool use**.

kv-store | Python, Raft, asyncio, Docker

GitHub | Blog

- Built a **distributed in-memory key-value store** (Python stdlib only) with a **3-node Raft** consensus layer — **leader election** via randomized timeouts, **log replication** with majority-quorum **AppendEntries**, and a **leader-isolation detector** that steps partitioned leaders down; integrated a **write-ahead log** with per-write **fsync** for crash recovery, deployed via **Docker Compose** with named volumes for **fault tolerance** under single-node failure.

CERTIFICATIONS & ACHIEVEMENTS

AWS Certified Cloud Practitioner | Amazon Web Services

Finalist | Qualcomm Edge AI Hackathon, New York